Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент Жаринов Михаил Андреевич

Группа ИУ7-42Б

Тип практики Технологическая

Название предприятия ООО «БауманСофт»

Студент _____ Жаринов М. А.
подпись, дата

Руководитель практики
от принимающей организации _____ Абрамова О. Л.
подпись, дата

Руководитель практики
от кафедры ИУ7 _____ Куров А. В.
подпись, дата

Рекомендованная оценка _____

Оценка _____

2025 г.

«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой _____
_____ _____
« ___ » _____ 20 __ г.

З А Д А Н И Е
на прохождение производственной практики
Технологическая

Студент Жаринов Михаил Андреевич 2 курса группы ИУ7-42Б

в период с 30.06.2025 г. по 20.07.2025 г.

Предприятие: ООО «БауманСофт»
Подразделение: основное

Руководитель практики от предприятия (наставник):
Абрамова Ольга Леонидовна, начальник отдела по работе с практикантами
Руководитель практики от кафедры:
Куров Андрей Владимирович, доцент кафедры, заместитель заведующего кафедрой ИУ7 МГТУ им. Н.Э. Баумана

Задание:
1. Проектирование, разработка и отладка программного кода прикладного и исследовательского характера на языке программирования C#

Дата выдачи задания « ____ » _____ 20___ г.

Руководитель практики от предприятия _____/ Абрамова О. Л. /

Руководитель практики от кафедры _____/ Куров А. В. /

Студент _____/ Жаринов М. А. /

Оглавление

Оглавление 3
1. Введение 4
1.1. Характеристика организации 4
1.2. Цели и задачи практики 4
1.3. Индивидуальное задание 5
2. Основная часть 6
2.1. Отчет о проделанной работе в соответствии с целями и задачами программы практики и индивидуальным заданием 6
2.2. Анализ и проектирование архитектуры системы 6
2.3. Описание веб-приложения 7
2.4. Реализация алгоритмов преобразования 8
Заключение 13
Список использованной литературы 14
Приложение 15

1. Введение

1.1. Характеристика организации

Общество с ограниченной ответственностью «БауманСофт» занимается разработкой программного обеспечения на заказной основе. Организация является многопрофильной, среди множества направлений можно выделить разработку интеллектуальных аналитических систем с использованием машинного обучения, анализа данных и применения больших языковых моделей. Также компания занимается прикладной разработкой информационно-аналитических и прикладных систем на языках программирования C#, Python, Golang, JavaScript, TypeScript, Rust, Kotlin и иных.

Предприятие реализует проекты различной степени сложности и выполняет весь цикл работ от предварительного обследования, анализа бизнес-процессов и составления технического задания, до разработки, внедрения и сопровождения конечных продуктов, поставки необходимого лицензионного ПО и серверного оборудования.

1.2. Цели и задачи практики

Цель: овладение умениями и навыками выполнения индивидуального задания на практику, умениями и навыками самостоятельной разработки технологий информационного анализа.

Задачи:

1. Овладение способностью выполнения анализа научно-технической информации и результатов исследований в области прикладной информатики.
2. Овладение способностью разработки технической и эксплуатационной документации информационных систем.
3. Изучение правил, регламентов и общих принципов организации аналитической работы в организации прохождения практики.
4. Формирование навыка поиска информации и работы с информационными материалами.

5. Разработка алгоритмов либо программного обеспечения для решения аналитических задач.

1.3. Индивидуальное задание

– Ознакомление с техникой безопасности и информационной политики в организации

– Выполнение задач организации по разработке и отладке программно-аналитических компонентов на языке программирования C#

– Разработка серверной части веб-приложения, позволяющего получить описание на русском языке введенной cron-строки формата Quartz.NET, а также получить по введенному описанию соответствующую cron-строку.

2. Основная часть

2.1. Отчет о проделанной работе в соответствии с целями и задачами программы практики и индивидуальным заданием

В рамках практики выполнялись задачи по командной разработке и отладке программно-аналитических компонентов. Задачи включали в себя анализ и проектирование архитектуры системы, определение списка требований для минимально жизнеспособного продукта (MVP), определение используемых технологий, реализация MVP. Были задействованы инструменты: язык C# и фреймворк ASP.NET для написания backend-части, библиотеки CronExpressionDescriptor и Quartz.NET для реализации расшифровки cron-строк, система контроля версий git, nginx и docker для развертывания приложения на удаленном сервере, большая языковая модель Gemini 2.5 Pro в целях оптимизации временных ресурсов решения задач.

В качестве среды разработки использовалась Visual Studio.

2.2. Анализ и проектирование архитектуры системы

На этапе анализа задачи были разработаны следующие требования к разрабатываемому программному продукту:

– Реализовать возможность составлять крон-строку по описанию на естественном (русском) языке

– Реализовать возможность расшифровывать крон-строку в ее описание на естественном (русском) языке

– Реализовать адаптивность интерфейса сайта для мобильных устройств

– Реализовать хранение последних операций в веб-приложении

Были разработаны UML-диаграммы архитектуры системы: диаграмма компонентов (рисунок 1) и диаграмма классов (рисунок 2)

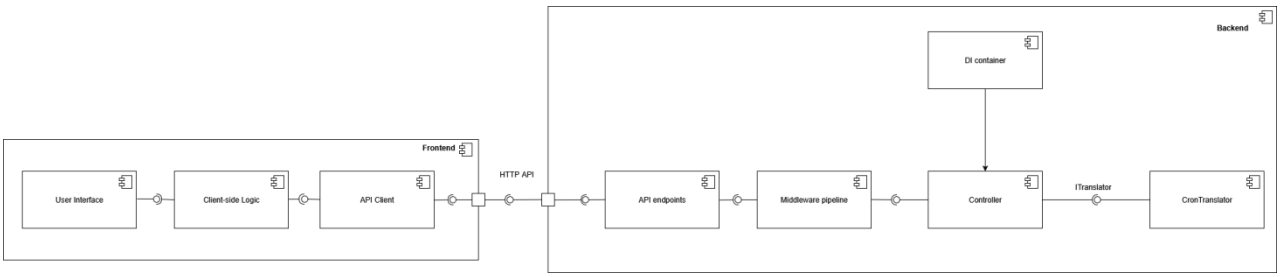

Рисунок 1 – UML-диаграмма компонентов

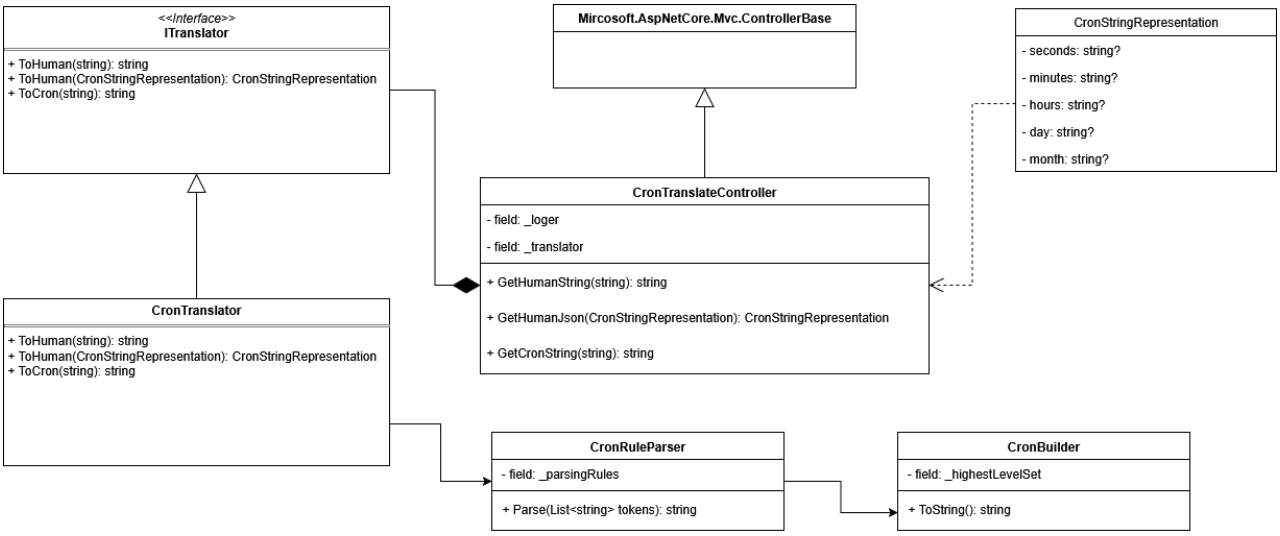

Рисунок 2 – UML-диаграмма классов

2.3. Описание веб-приложения

В ходе командной разработки были получены следующие результаты:

– Реализована верстка веб-сайта, представляющую пользователю необходимую функциональность, внешний вид которого представлен в Приложении на рисунке 1.

– Реализована отправка запросов с клиента на сервер с использованием REST API.

– Реализован прием и первичная обработка запросов на стороне сервера с помощью фреймворка ASP.NET.

– Реализован вызов методов ToHuman и ToCron через интерфейс ITranslator, описание которого приведено на Листинге 1, что позволяет при необходимости менять реализацию алгоритма преобразования.

```
    public interface ITranslator
    {
        public string ToHuman(string expression);
        public string ToHuman(CronStringRepresentation representation);
        public string ToCron(string description);
    }
```

Листинг 1 – Описание интерфейса ITranslator

Моя задача состояла в реализации класса, предоставляющего интерфейс ITranslator, то есть в разработке алгоритмов преобразования описания в cron-строку и обратно.

2.4. Реализация алгоритмов преобразования

Благодаря наличию готовой библиотеки CronExpressionDescriptor, задача преобразования cron-строки в ее описание решается несложно и состоит в следующем:

– Первичная проверка входной cron-строки на корректность с помощью библиотеки Quartz.NET

– Попытка получения описания с помощью метода GetDescription библиотеки CronExpressionDescriptor

– Проверка успешности получения описания

– Возврат полученного описания

Реализация метода ToHuman приведена в Листинге 2

```
        public string ToHuman(string expression) {
            if (!CronExpression.IsValidExpression(expression)) {
                throw new Exceptions.CronException("Некорректный синтаксис крон-
строки!");
            }
            string desc;
            try
            {
                desc = ExpressionDescriptor.GetDescription(expression, new
Options()
                {
                    DayOfWeekStartIndexZero = false,
                    Use24HourTimeFormat = true,
                    Locale = "ru"
                });
            }
            catch (FormatException) {
                throw new Exceptions.CronException("Некорректный синтаксис крон-
строки!");
            }
            return desc;
        }
```

Листинг 2 – Реализация метода ToHuman

Получение cron-строки по ее описанию на русском языке является более сложным и состоит из следующих этапов:

– Токенизация строки

– Нормализация всех слов (перевод в начальную форму)

– Поиск в строке шаблонов, описывающих часто встречающиеся в русском языке конструкции для обозначения частоты выполнения некоторого действия, например, *каждую* [число] *секунду*.

– Получение итоговой cron-строки

– Проверка выходной cron-строки на корректность с помощью библиотеки Quartz.NET

Процесс токенизации состоит в приведении строки к нижнему регистру и разбиению строки на слова по пробелам.

Нормализация заключается в замене всех запятых на союз «и» и замене каждого слова на его начальную форму через словарь всех релевантных слов и всех их форм.

Реализация класса, позволяющего выполнить токенизацию и нормализацию с помощью метода Lemmify, приведена в Листинге 3.

```
    public static class CronLemmificator
    {
        private static readonly Dictionary<string, string> WordMap;
        static CronLemmificator()
        {
            WordMap = [];
            AddMapping("секунда", "секунды", "секунду", "секунд");
            AddMapping("минута", "минуту", "минуты", "минут");
            …
            AddMapping("каждый", "каждую", "каждое", "каждые");
        }
        private static void AddMapping(string canonical, params string[]
variations)
        {
            WordMap[canonical] = canonical;
            foreach (var variation in variations)
            {
                WordMap[variation] = canonical;
            }
        }
        private static string NormalizeWord(string word)
        {
            if (WordMap.TryGetValue(word, out var canonicalForm)) {
                return canonicalForm;
            }
            return word;
        }
```

```
        public static List<string> Lemmify(string phrase)
        {
            string cleanPhrase = phrase.ToLower();

            string[] tokens = cleanPhrase.Split([' ', ';'],
StringSplitOptions.RemoveEmptyEntries);

            var normalizedTokens = new List<string>();
            foreach (var token in tokens)
            {
                bool isComma = false;

                string normalized = token;
                if (normalized.EndsWith(','))
                {
                    normalized = normalized[..^1];
                    isComma = true;
                }
                normalized = NormalizeWord(normalized);
                normalizedTokens.Add(normalized);
                if (isComma)
                {
                    normalizedTokens.Add("и");
                }
            }

            return normalizedTokens;
        }

    }
```

Листинг 3 –Реализация класса CronLemmificator

Для поиска в нормализованной строке необходимых шаблонов, используется следующий принцип: в классе CronRuleParser создается список делегатов (функций), имеющих одинаковую сигнатуру – они принимают на вход список токенов, индекс текущего слова в списке, и обьект класса CronBuilder, позволяющий легко менять параметры текущего состояния cron-строки. Эти функции пробуют применить шаблон, который они применяют, к текущей позиции в списке токенов, и, если это получается, то изменяют соответствующим образом CronBuilder и возвращают количество токенов в примененном шаблоне, иначе возвращают 0. Эти функции по очереди в цикле применяются к списку токенов, в случае, если ни один шаблон не подходит, то индекс смещается на 1, иначе – на количество подошедших токенов. Реализация применения шаблонов приведена в Листинге 4.

```
    public class CronRuleParser
    {
        private delegate int RuleParser(List<string> tokens, int index,
CronBuilder builder);
        private static readonly List<RuleParser> _parsingRules;

        static CronRuleParser()
        {
            _parsingRules =
            [
                TryParseFullTime,
                TryParseTime,
                TryParseLastDayOfMonth,
                TryParseNthDayOfWeek,
                TryParseRange,
                TryParseSpecificValues,
                TryParseInterval,
                TryParseEvery,
            ];
        }

        public static string Parse(List<string> tokens)
        {
            var builder = new CronBuilder();
            for (int i = 0; i < tokens.Count;)
            {
                int consumedTokens = TryParsePatterns(tokens, i, builder);
                if (consumedTokens > 0)
                {
                    i += consumedTokens;
                }
                else
                {
                    i++;
                }
            }
            return builder.ToString();
        }

        private static int TryParsePatterns(List<string> tokens, int index,
CronBuilder builder)
        {
            foreach (var rule in _parsingRules)
            {
                int consumed = rule(tokens, index, builder);
                if (consumed > 0)
                {
                    return consumed;
                }
            }

            return 0;
        }
```

Листинг 4 – Реализация логики применения функций поиска шаблонов

Пример функции, которая пытается применить шаблон «в последний день» приведен в Листинге 5.

```
        private static int TryParseLastDayOfMonth(List<string> tokens, int
index, CronBuilder builder)
        {
            if (index + 2 < tokens.Count &&
                tokens[index] == "в" &&
                tokens[index + 1] == "последний" &&
                tokens[index + 2] == "день")
            {
                builder.DayOfMonth = "L";

                if (index + 3 < tokens.Count &&
MonthsMap.ContainsKey(tokens[index + 3]))
                {
                    builder.Month = MonthsMap[tokens[index + 3]].ToString();
                    return 4;
                }

                return 3;
            }

            return 0;
        }
```

Листинг 5 – Реализация функции, применяющей один из шаблонов

Для автоматизированной проверки реализованных мной алгоритмов были написаны тесты с использованием фреймворка MSTest.

Заключение

В ходе прохождения производственной практики в ООО «БауманСофт» были выполнены все цели и задачи. В результате командной разработки мы смогли реализовать MVP продукта. Мы разработали сервис, позволяющий составлять cron-строку по описанию на русском языке, а также получать описание cron-строки на русском языке.

В результате практики я изучил создание backend-части веб-приложения на C#, фреймворк MSTest, основы REST API и Nginx, а также обработку естественных языков (NLP) . Также я получил опыт в командной разработке.

Список использованной литературы

1. Полное руководство по языку программирования C# 13 и платформе .NET 9 [Электронный ресурс]. – URL: https://metanit.com/sharp/tutorial/ (дата обращения: 04.07.2025)

2. Quartz.NET Documentation [Электронный ресурс]. – URL: https://www.quartz-scheduler.net/ (дата обращения: 18.07.2025)

Приложение

Рисунок 1 – Внешний вид веб-сайта